Komori Project Specification

Cross Platform Development | WS2026 | Group G | 28 September 2026

2.1 Project Introduction

Team: Polini Yann and Traxler Diego. **Working title:** Komori.

Komori helps viewers find something to watch and remember which titles they started or finished. It offers a searchable catalog, personal lists, ratings and recommendations. Casual anime viewers are the target group. They prefer tracking a whole title or season to updating an episode counter.

The first version covers catalog browsing, title details and ratings, three watch statuses, custom lists, genre preferences, recommendations, accounts, reminders and offline editing. Tracking follows the provider's catalog entries. A season with its own entry gets its own tracking, and Komori adds no season groupings. This version excludes streaming, shared lists, episode tracking and ratings, ratings for entire franchises, year/month filters and non-anime TV or movies.

2.2 Chosen Framework

We chose **React Native with Expo and TypeScript**. Reusable components keep the interface consistent, and TypeScript catches mistakes in the data passed between them. Expo provides tools for navigation, storage and notifications. We start from the official FHTW-CPD React Native/Expo template and keep package versions compatible with it. Android is the main platform for development and testing.

2.3 Planned Features (User Stories)

Eight stories define the main features and the acceptance criteria for the Android app. Fixed sample responses make API tests repeatable. Paths in the Implemented in lines are relative to src/, except the Supabase migration, which is relative to the repository root.

US-01 Discover anime

As an anime viewer, I want to search and browse titles and releases, so that I can find something to watch.

This is done when: (a) Discover combines title search and a genre filter, with sorting by popularity, catalog score or title. Changing a filter or sort order resets the list to page one. (b) Each page shows at most 12 cards. Opening a card shows Details, and Back restores the search, filters and page. (c) New releases shows known premiere dates from the last 30 days through today. Upcoming shows future premieres, nearest first, and labels entries without a date as "Date unknown". (d) The screen shows loading, no-match and load-failure states, and offers Retry after a failure. Release feeds do not depend on personal preferences.

Implemented in: `app/(tabs)/discover.tsx`, `app/(tabs)/index.tsx`, `features/catalog/feeds.ts`, `features/catalog/useCatalog.ts`, `features/catalog/dates.ts`, `services/anilist.ts`, `components/AnimeCard.tsx`, `components/SearchField.tsx`, `components/ScreenState.tsx`

US-02 Inspect and rate a title

As a viewer, I want to read details and record my own rating, so that I can decide what to watch and remember my opinion.

This is done when: (a) Details shows the title, cover, synopsis, genres, release information and catalog score. Details marks missing information. (b) A signed-in user can set, change and clear a whole-number rating from 1 to 5. Cards and list rows show the updated rating. (c) Personal ratings (out of 5) and catalog scores (out of 10) stay apart. A rating changes neither watch status nor list membership. (d) Unreleased entries cannot be rated. If the release date is missing, the app uses the provider's airing status to check whether rating is allowed.

Implemented in: `app/anime/[id].tsx`, `components/Rating.tsx`, `features/library/rules.ts` (setRating, canRate), `features/catalog/dates.ts` (isReleased)

US-03 Maintain watch status

As a viewer, I want to organize titles as Plan to watch, Watching or Watched, so that I can keep track without counting episodes.

This is done when: (a) A title has one watch status at a time. Changing it moves the title between the built-in lists and updates counts across the app. (b) Quick-save adds an untracked title to Plan to watch and leaves the status of a tracked title unchanged. (c) Removing a watch status leaves ratings and custom lists unchanged. (d) Watched acts as the watch history. Each built-in list supports title search and links to Details. Removing a status asks for confirmation.

Implemented in: `app/anime/[id].tsx`, `app/(tabs)/lists.tsx`, `app/list/[id].tsx`, `features/library/rules.ts`, `features/library/reducer.ts`, `features/library/LibraryProvider.tsx`, `components/StatusBadge.tsx`

US-04 Organize custom lists

As a viewer, I want personal themed lists, so that I can group titles apart from my watch progress.

This is done when: (a) Lists shows built-in and custom lists together, with counts and a search by list name. (b) Users can create, rename and delete custom lists. Names must contain 1-40 characters after trimming spaces. The app rejects duplicate names regardless of capitalization, including names of built-in lists. (c) A title can belong to several custom lists and appears once in each. Adding or removing it leaves its status and rating unchanged. (d) Deleting a list asks for confirmation and removes that list and its memberships, nothing else. Empty lists offer an add/browse action.

Implemented in: `app/(tabs)/lists.tsx`, `app/list/[id].tsx`, `components/ListNameDialog.tsx`, `features/library/rules.ts` (validateListName, createList, moveToList)

US-05 Receive explainable recommendations

As a viewer, I want suggestions based on my genres and watched history, so that I can discover relevant titles.

This is done when: (a) Users select genres in Account, and Home updates its suggestions, up to four. (b) Suggestions exclude tracked and unreleased titles. (c) Among the available candidates, each preferred genre match adds 3 points and each watched title with a shared genre adds 2 points. The app sorts ties by catalog score, then catalog ID. Each suggestion shows the reason for its selection. (d) Popular titles fill the remaining slots when there are no matches. If no suitable titles remain, the app says so. Personal ratings do not affect recommendations in this version.

Implemented in: `app/(tabs)/index.tsx`, `app/account.tsx` (genre selection), `features/recommendations/scoring.ts`, `features/recommendations/candidates.ts`, `features/recommendations/useRecommendations.ts`

US-06 Use a personal account

As a viewer, I want to sign in to my own account, so that my lists, ratings and preferences remain separate and available on another device.

This is done when: (a) Users can register online with an email and password, verify their email, sign in and sign out through the account service. Invalid credentials and network errors show clear messages. (b) Account shows an editable display name, preferences, counts and up to six watched titles. See all opens Watched. (c) Signing in on another device loads the latest synced data. Other accounts cannot access it. (d) Sign-out clears the local session and private data and cancels reminders. If changes are waiting to sync, the user must choose Sync first, Discard or Cancel.

Implemented in: `app/account.tsx`, `features/account/AuthForm.tsx`, `features/account/AuthProvider.tsx`, `features/account/client.ts`, `features/account/sessionStorage.ts`, `features/account/authStorage.ts`, `features/account/signOut.ts`, `supabase/migrations/20261107180000_komori_profiles.sql`

US-07 Set a watch reminder

As a viewer, I want a local reminder for a saved title, so that I remember to return to it.

This is done when: (a) Details lets users set one reminder per watch-listed title at a future time. The app asks for permission when the user chooses this feature and not before. (b) A notification appears on the Android test device with the app in the background and the network off. Tapping it opens the saved Details screen for that title. (c) Editing replaces the old reminder. Canceling it or removing the watch status cancels the notification. (d) If permission is denied or scheduling fails, the app shows "Not scheduled". Tracking keeps working, and the app offers a link to system settings when appropriate.

Implemented in: `features/reminders/ReminderControls.tsx`, `features/reminders/ReminderProvider.tsx`, `features/reminders/reminders.ts`, `features/reminders/platform.ts`, `app/anime/[id].tsx`

US-08 Keep working offline and recover safely

As a returning user, I want my saved data to remain usable offline, so that a lost connection does not stop me tracking anime.

This is done when: (a) After an online session, saved details, lists, ratings and preferences remain available after restarting in airplane mode. (b) Offline edits survive another restart and show Pending sync. (c) Reconnecting uploads pending changes and refreshes data without duplicate list entries. Changes made on another device trigger the conflict choice in section 2.7. (d) Searches without cached results explain that internet is needed. Failed refreshes keep existing data. A new sign-in requires internet.

Implemented in: `storage/sqliteRecords.ts`, `storage/store.ts`, `features/library/localRepository.ts`, `features/catalog/cache.ts`, `services/connectivity.ts`, `features/library/LibraryNotice.tsx`, `features/sync/sync.ts`, `features/sync/SyncProvider.tsx`, `features/sync/cloudProfiles.ts`

2.4 Screen / Navigation Structure

The app will have **Home / Discover / Lists** bottom tabs and Account in the header. The logo returns Home. Details and subpages use stack navigation. Android Back preserves the browsing state.

Screen	Content and actions	Required state
Home	Search, up to four suggestions, three-title previews for New releases and Upcoming, See all and genre shortcuts	Recommendation candidates, preferences/history, cached feeds, loading/errors
Discover and release pages	Cover cards, title search, genre filter, sorting and pages. Cards open Details	Mode, query, genre, sort and page in the route, cached result IDs
Title Details	Title information, personal rating, watch status, custom lists and reminder controls	Catalog ID, cached details, account selections, permission/reminder status
Lists overview	Built-in and custom lists, counts, name search and list editing	Profile data, search text, editing dialog
List Detail	Titles in alphabetical order, search, open, move and remove actions	List ID, query, memberships, selected action
Account and sign-in	Sign-in, display name, genres, history preview and sync status	Session, profile, form inputs, pending changes/conflicts

Main routes: Home search or See all -> Discover/release page -> Details -> watch status/custom list -> Lists -> List Detail -> Details. Header -> Account -> Watched history -> Details. A notification opens saved Details, or Home if the title/account is no longer available.

Design: Off-white backgrounds, green buttons, rounded cards and outline icons give the screens a consistent look. API covers have placeholders. Buttons use labels and 44-point touch targets, and long lists use virtualized rendering. Layout checks cover 360- and 390-point widths, larger text, keyboard input and screen-reader labels. Status uses text and color.

2.5 API and Data Sources

Anime data: The AniList GraphQL API (POST `https://graphql.anilist.co`) is the catalog source. It is public and read-only, needs no login or API key, and provides IDs, titles, cover images, synopses, genres, start/end dates, airing status and average scores (shown as a score out of 10). Personal lists and ratings stay separate from AniList. Jikan proved unreliable and unmaintained during development, so the team moved to AniList. Komori stores AniList IDs. [1]

Request	Use in Komori
`Page(page, perPage: 12)` { media(type: ANIME, isAdult: false, search, genre, sort) }	Discover. Sort by `POPULARITY_DESC` for popularity, `SCORE_DESC` for rating or `TITLE_ROMAJI for title`
`Media(id)` and `GenreCollection`	Title details and genre choices
`media(sort: START_DATE_DESC, startDate_greater, startDate_lesser)`	Recent releases. Check returned premiere dates against the 30-day window
`media(status: NOT_YET_RELEASED, sort: START_DATE)`	Upcoming titles. Separate known and unknown dates

Date checks may remove results. Load more keeps API paging available until the app has loaded all pages. New releases excludes unknown dates. Recommendations use up to 100 cached candidates from popular titles and up to three preferred genres.

Accounts: The proposed setup combines Supabase Auth for email/password sign-in with a private `profiles` table for personal data and its revision number. Owner access policies keep each account's data private. A save operation checks the revision before updating it. The app uses a publishable key, with no privileged keys on the device. [2]

Failed requests: Search waits 400 ms after typing. The API client ignores outdated responses, avoids duplicate requests and limits calls to one per second. The client handles AniList rate limits (HTTP 429 with Retry-After) with increasing waits and limited retries. Manual Retry keeps cached results. Missing text/images use placeholders. Labeled sample data never replaces failed live results without notice.

2.6 Native Feature Usage

Local Android notifications will provide optional watch reminders through `expo-notifications`. [3] In a saved title's Details screen, the user chooses a future date and time. The app stores the title ID, reminder time and notification ID. Reminders use a dedicated notification channel. The app asks for permission when the user first sets a reminder. Tapping the notification opens that title once its saved data loads.

Reminders help users remember what they wanted to watch. They show neither a broadcast time nor streaming availability. Each account can have one reminder per title on each device. Editing a reminder replaces the previous schedule. Canceling it, removing the watch status or signing out clears it. Failed scheduling stays visible, and the user can retry it. Reminders stay on the device that created them and do not sync to other devices.

If permission is denied or scheduling is unavailable, the rest of the app stays usable. The reminder controls show the permission status, explain the issue and offer system settings. The app does not re-prompt for permission. Android's battery restrictions can delay notifications, so delivery at the exact minute is not guaranteed. At startup, the app checks saved reminders against the device's scheduled notifications. We will test scheduling, opening, editing, cancellation and denied permission on an Android device or emulator.

2.7 Offline / Persistence Strategy

Offline storage will use `expo-sqlite` for app data and a tested `expo-secure-store` adapter for session tokens. The sign-in form uses passwords, and the app never saves them. SQLite keeps local data available after a restart. [4] The app also works without an account. The guest library stays on the device, is separate from account libraries and is not synced.

Stored locally	Why we store it and for how long
Private profile: display name, preferred genre names (up to three), watch status by AniList ID, ratings, custom lists and memberships	Enables offline use. Stays for the current account until sign-out or explicit deletion
Sync information: server revision, pending local revision and last successful sync	Helps recover interrupted saves and detect changes made on another device
Saved catalog entries by AniList ID, query pages, genres and timestamps	Keeps saved title information offline. Refresh after 24 hours when opened online. Remove unused query cache after seven days
Reminder records and notification IDs	Restores reminder controls and checks the device schedule. Not included in cloud sync
Session tokens in secure storage	Restores the session, refreshes it online and clears it on sign-out. The app writes tokens to neither logs nor the catalog cache

Saving changes: The app saves each personal edit in a local transaction. A successful save updates the shared state and marks the profile Pending sync. If saving fails, the app shows an error and never reports success. The app keeps title information for anything a watch status, list, rating or reminder references. The app shows cached covers offline and uses placeholders when an image is unavailable.

Synchronizing: Signed in and online, the app syncs after edits, on reopening, on reconnecting and when the user selects Retry. It uploads the saved profile with the expected server revision. The server rejects the upload when the revision no longer matches. Otherwise it accepts the upload and returns the new revision. Edits the user makes during an upload stay pending for the next save. Failed uploads keep local data. If nothing is pending, the app loads the current cloud version. An expired session pauses uploads. The app asks for sign-in and keeps local work available.

Changes on two devices: If the cloud revision changed while local edits are pending, the app keeps the local draft and stops automatic replacement. It explains the conflict and offers Keep this device or Use cloud, with confirmation before discarding a version. If the cloud changes again, the app checks the revision again. The project excludes automatic merging and collaborative editing.

Offline limits: Users who signed in earlier can read and edit saved data offline. New sign-ins, uncached searches and updates from another device need internet. The app shows Offline, Pending sync, Sync failed or Up to date. Signing out with pending changes follows US-06. The app clears account data before it shows another account, while public catalog data can stay cached.

2.8 Technical Approach & Architecture

We organize the code into **screens, hooks/context, and services/repositories**. Expo Router handles navigation. React Context and a reducer share the profile and update actions across Home, Details, Lists and Account. Form inputs stay within their screens. Rules for lists, ratings and recommendations are pure TypeScript functions, testable in isolation. Screens call services and repositories, never APIs or storage.

Data flow: Screen -> hook/action -> validation -> repository -> SQLite -> shared state -> screen. Catalog and sync services communicate with AniList and Supabase. A reminder service handles notifications. The app loads saved data at startup before it shows personal lists, so an empty initial state cannot overwrite existing data.

Planned location	Purpose and related stories
`src/app/`	Tab/stack navigation and all screens. US-01 to US-08
`src/components/`	Cards, list rows, rating input, filters, loading/error messages and shared controls
`src/features/catalog/`	Search/release hooks, catalog model and request status. US-01, US-02
`src/features/library/`	Watch status, list and rating rules, shared state and actions. US-02, US-03, US-04
`src/features/recommendations/`	Candidate selection, scoring and explanations. US-05
`src/features/account/`	Sign-in/session handling, profile editing and conflict choices. US-06, US-08
`src/features/sync/`	Cloud sync and conflict handling. US-06, US-08
`src/features/reminders/`	Local watch reminders. US-07
`src/services/`	Catalog client and connectivity check. US-01, US-08
`src/storage/` and `supabase/migrations/`	Local tables, migrations and repositories. Cloud profile table, owner policies and revision checks
`__tests__` folders next to the code they test	Tests for business rules, components, storage and services, e.g. src/features/library/__tests__/rules.test.ts

Main data types: `Anime` contains the AniList ID and available title information. `ProfileData` holds the schema version, display name, preferred genre names, watch statuses, ratings and custom lists. Lists have stable UUIDs and cannot contain duplicate anime IDs. `ProfileRecord` adds the owner ID and server revision. `Reminder` links the owner and title to a device schedule. Shared validation checks rating limits, one watch status per title, valid list names and unique memberships. The server checks ownership and revisions.

Testing: 73 automated tests in 8 suites cover (1) watch status and independent list/rating changes, (2) recommendation scoring, exclusions and date boundaries, (3) API success, empty results, errors and rate limits, (4) saved edits after reload and interrupted/conflicting sync, and (5) account separation and notification denial/cancellation using service mocks. Type checking (`npm run typecheck`) and tests (`npm test`) run locally before each pull request. CI and linting were not set up in this version. We will also walk through all eight stories on Android, including real sign-in, restarting in airplane mode and notifications.

Implementation by user story

Paths are relative to src/, except the migration, which is relative to the repository root.

Story	Implemented in
US-01 Discover anime	`app/(tabs)/discover.tsx`, `app/(tabs)/index.tsx`, `features/catalog/feeds.ts`, `features/catalog/useCatalog.ts`, `features/catalog/dates.ts`, `services/anilist.ts`, `components/AnimeCard.tsx`, `components/SearchField.tsx`, `components/ScreenState.tsx`
US-02 Inspect and rate a title	`app/anime/[id].tsx`, `components/Rating.tsx`, `features/library/rules.ts` (setRating, canRate), `features/catalog/dates.ts` (isReleased)
US-03 Maintain watch status	`app/anime/[id].tsx`, `app/(tabs)/lists.tsx`, `app/list/[id].tsx`, `features/library/rules.ts`, `features/library/reducer.ts`, `features/library/LibraryProvider.tsx`, `components/StatusBadge.tsx`
US-04 Organize custom lists	`app/(tabs)/lists.tsx`, `app/list/[id].tsx`, `components/ListNameDialog.tsx`, `features/library/rules.ts` (validateListName, createList, moveToList)
US-05 Receive explainable recommendations	`app/(tabs)/index.tsx`, `app/account.tsx` (genre selection), `features/recommendations/scoring.ts`, `features/recommendations/candidates.ts`, `features/recommendations/useRecommendations.ts`
US-06 Use a personal account	`app/account.tsx`, `features/account/AuthForm.tsx`, `features/account/AuthProvider.tsx`, `features/account/client.ts`, `features/account/sessionStorage.ts`, `features/account/authStorage.ts`, `features/account/signOut.ts`, `supabase/migrations/20261107180000_komori_profiles.sql`
US-07 Set a watch reminder	`features/reminders/ReminderControls.tsx`, `features/reminders/ReminderProvider.tsx`, `features/reminders/reminders.ts`, `features/reminders/platform.ts`, `app/anime/[id].tsx`
US-08 Keep working offline and recover safely	`storage/sqliteRecords.ts`, `storage/store.ts`, `features/library/localRepository.ts`, `features/catalog/cache.ts`, `services/connectivity.ts`, `features/library/LibraryNotice.tsx`, `features/sync/sync.ts`, `features/sync/SyncProvider.tsx`, `features/sync/cloudProfiles.ts`

2.9 Changelog

Date	Change
24 Sep 2026	Discussed the project and agreed on the app's scope and technical approach.
28 Sep 2026	Wrote the initial project specification, including the user stories, screen structure and technical plan.
7 Nov 2026	Switched the catalog source from Jikan to AniList and aligned the data model and guest mode with the implementation.
12 Nov 2026	Final submission: linked each user story to its implementation, corrected the test location, CI note and folder table.

References: This specification follows the Moodle Project Requirements, sections 2.1-2.9. Technical references: [1] AniList GraphQL API documentation (docs.anilist.co). [2] Supabase React Native Auth and owner access policies. [3] Expo notifications. [4] Expo SQLite and SecureStore.